Module:5

Memory Management

Module:5	Memory Management	7 hours
Main memory management, Memory allocation strategies, Virtual memory: Hardware support for virtual memory (caching, TLB) – Paging - Segmentation - Demand Paging - Page Faults - Page Replacement -Thrashing - Working Set.		

Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

- Divide physical memory into fixed-sized blocks called **frames**
- Divide logical memory into blocks of same size called **pages**

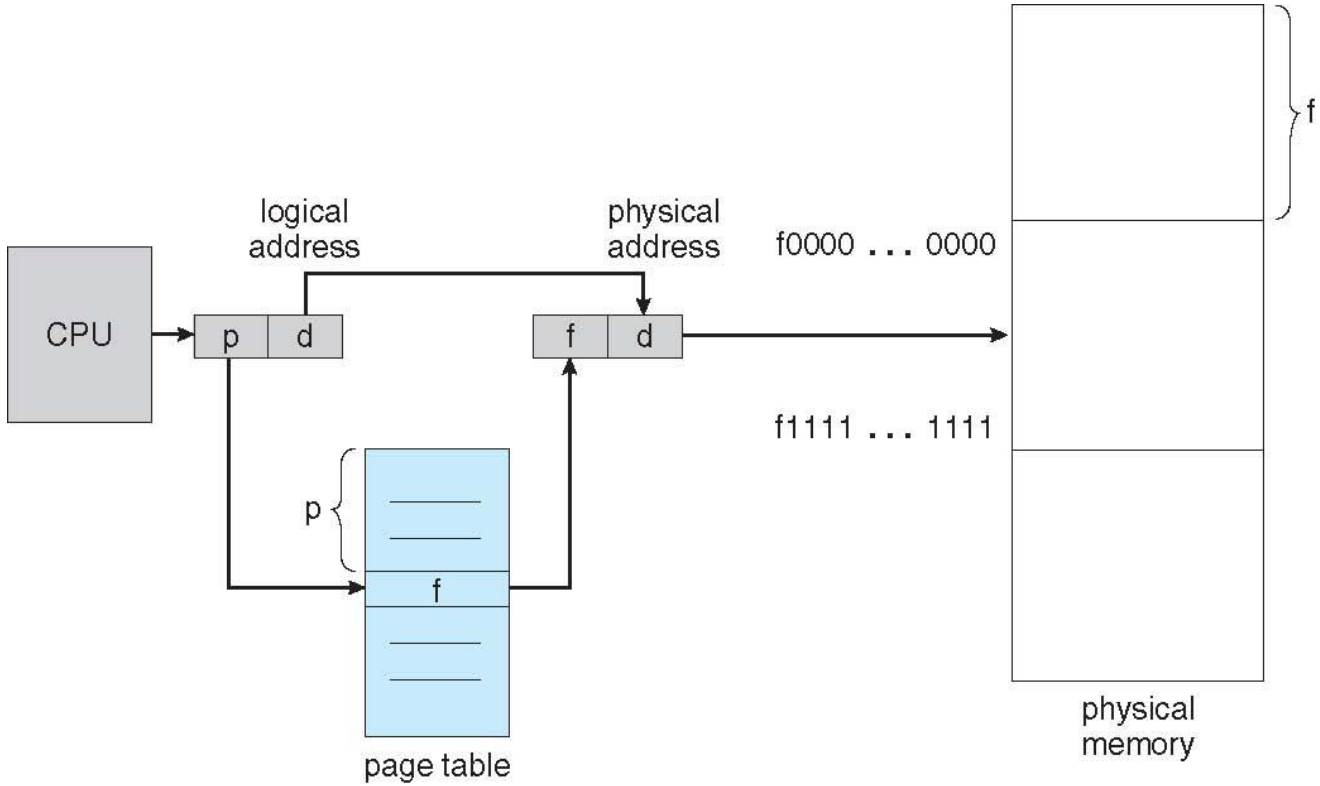
Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit

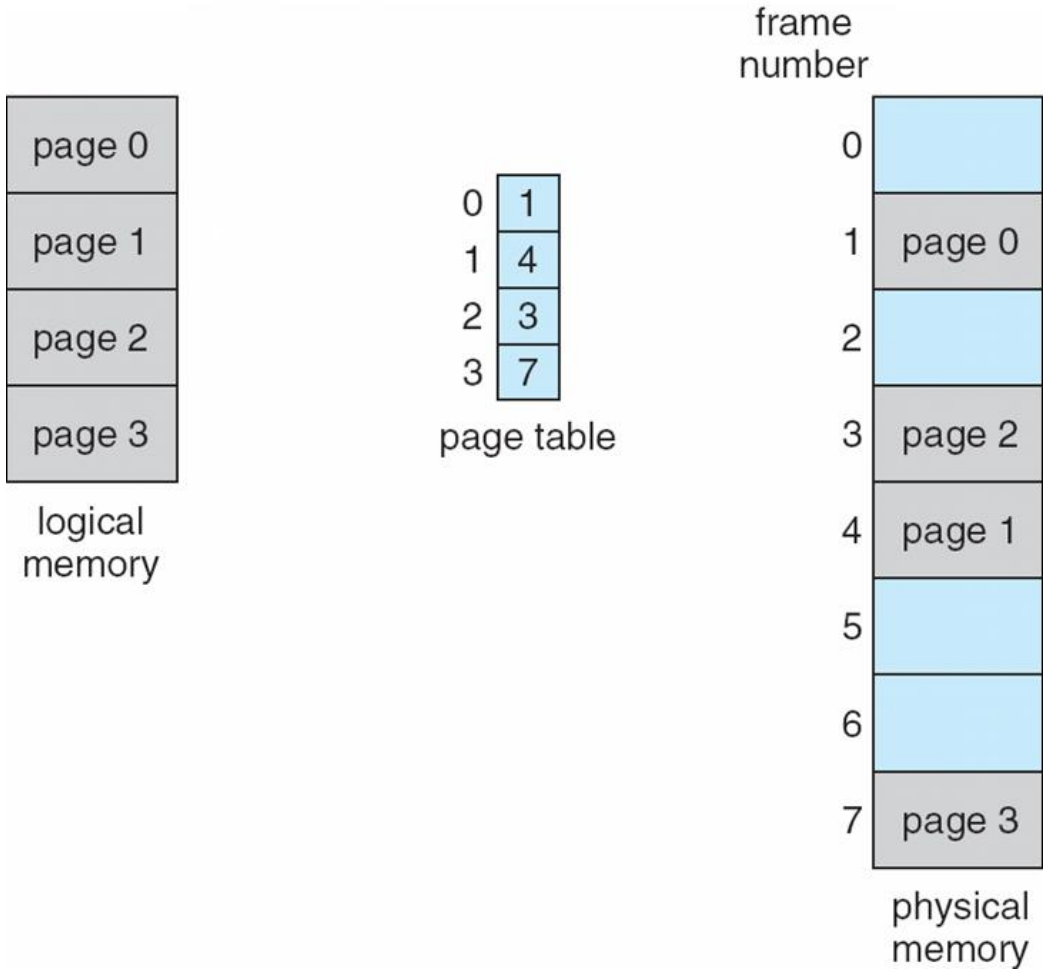
page number	page offset
p	d
m -n	n

  - For given logical address space 2^m and page size 2^n

Paging Hardware

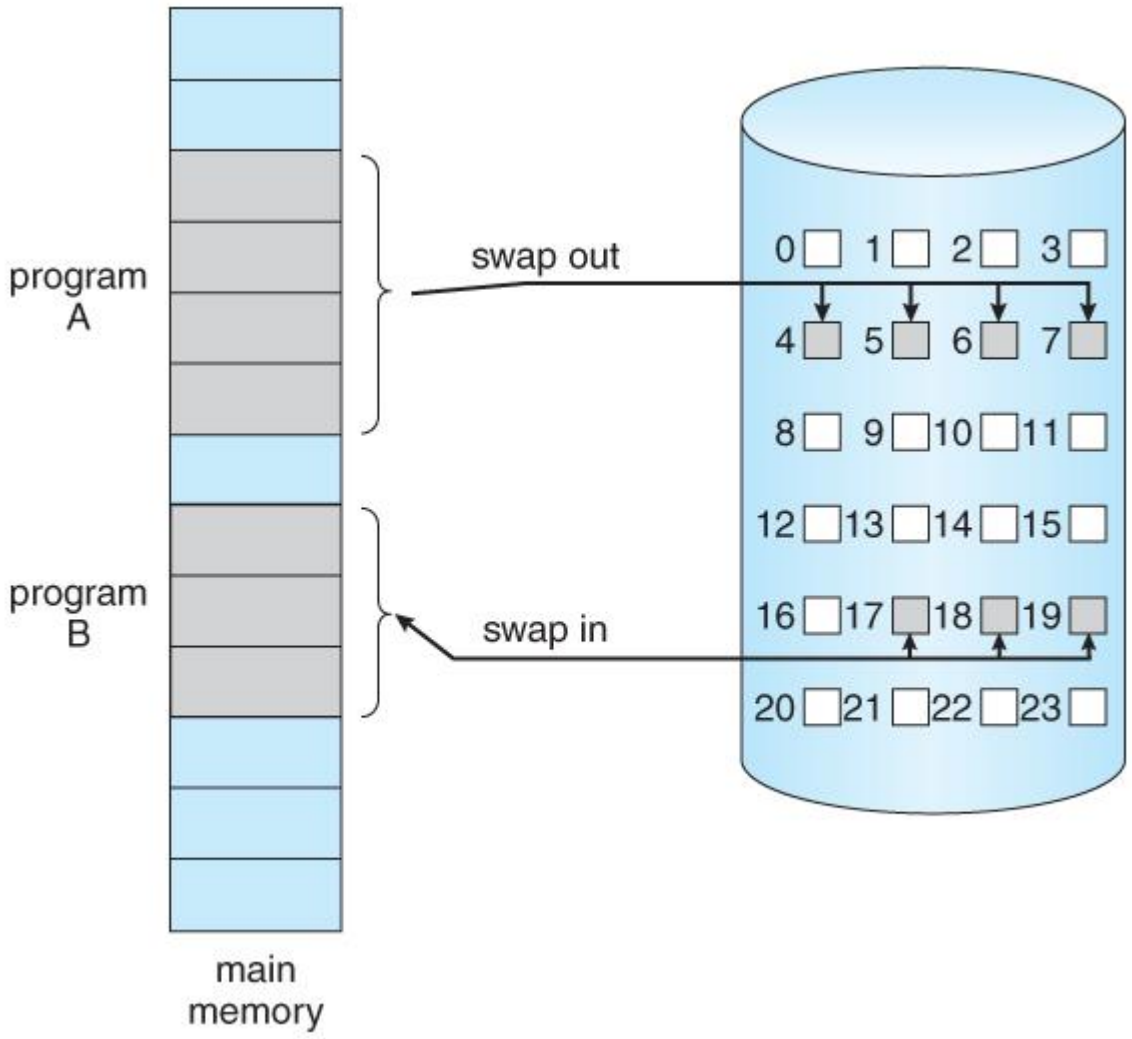


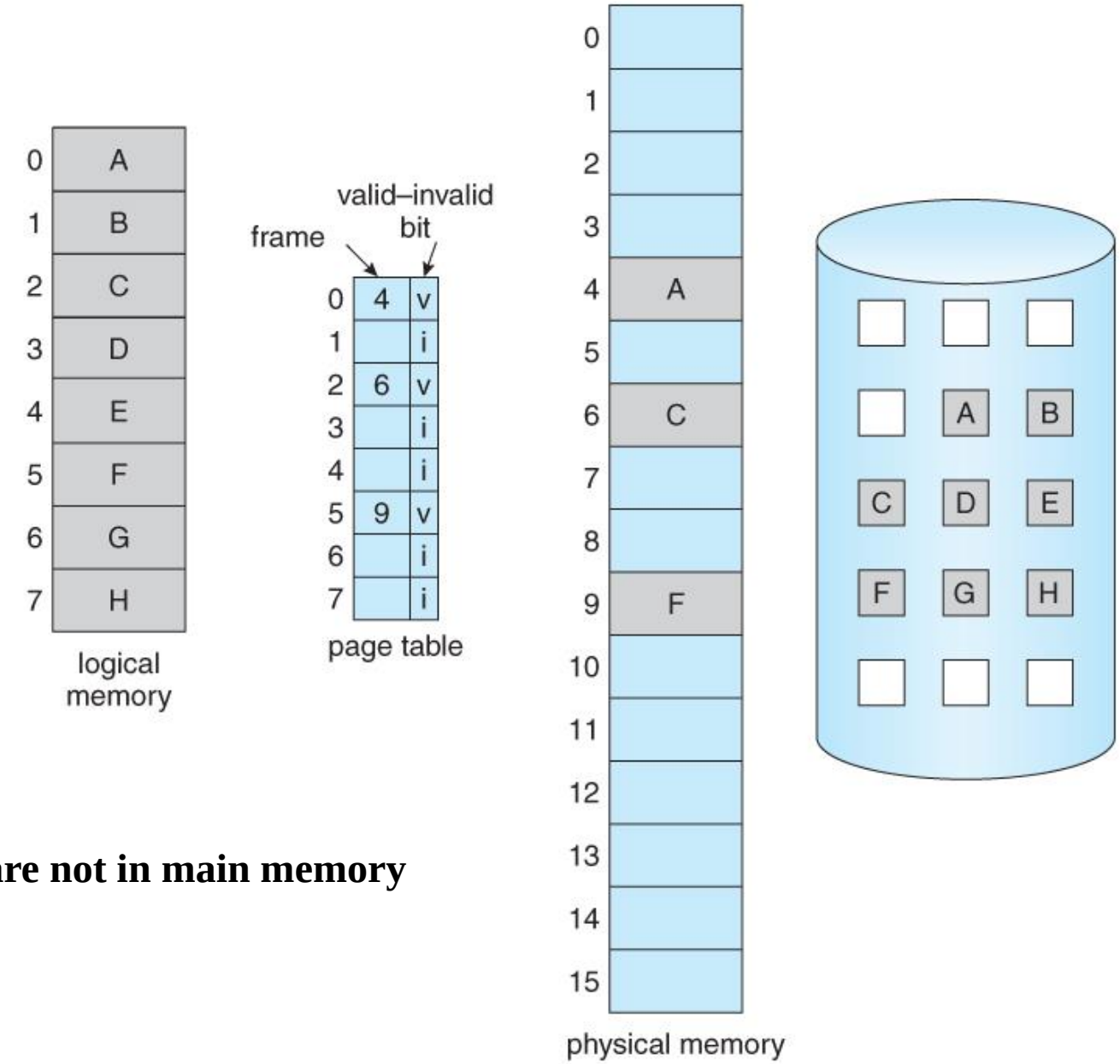
Paging Model of Logical and Physical Memory



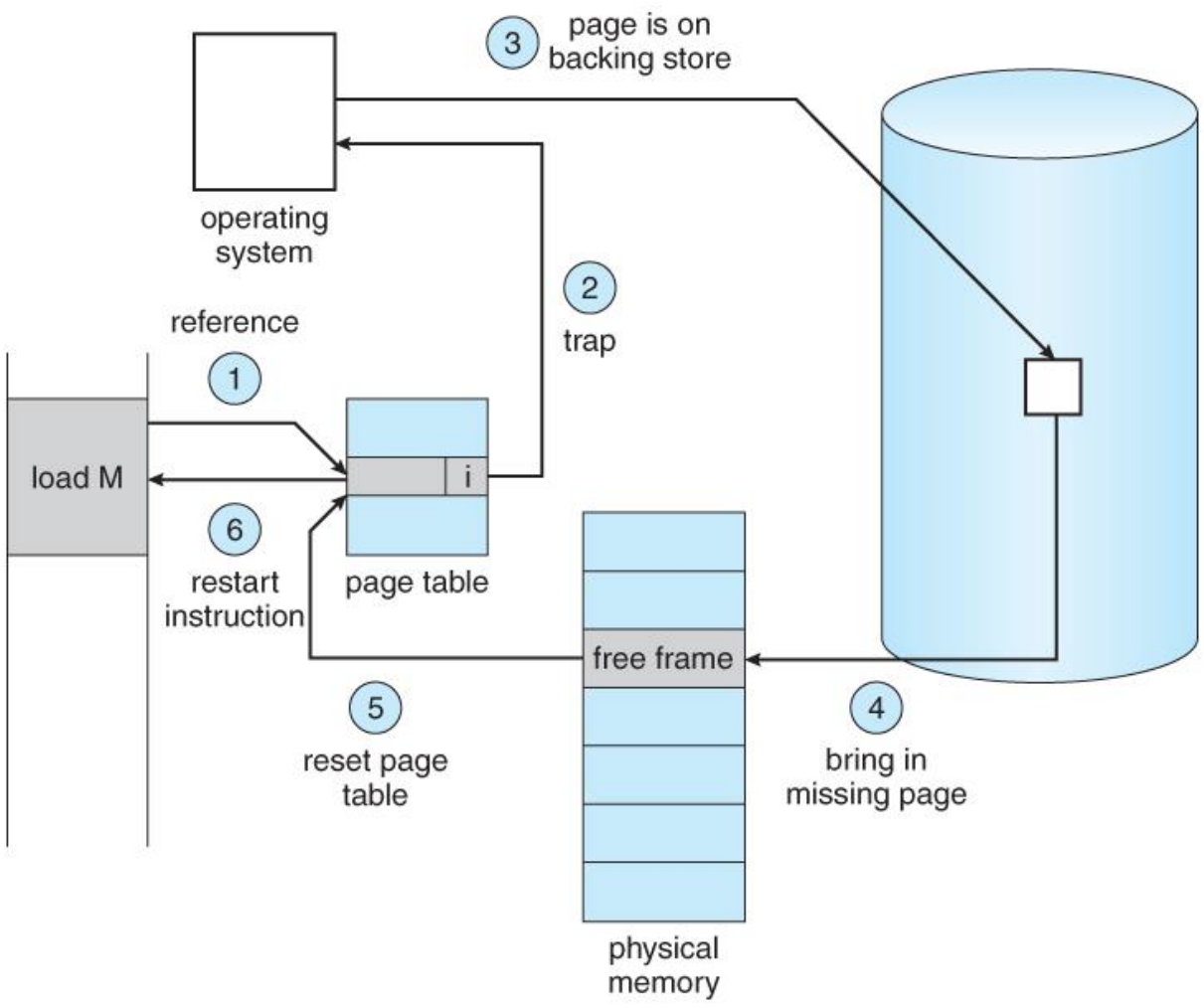
Demand Paging

- According to the concept of Virtual Memory, in order to execute some process, only a part of the process needs to be present in the main memory which means that only a few pages will only be present in the main memory at any time.
- However, deciding, which pages need to be kept in the main memory and which need to be kept in the secondary memory, is going to be difficult because we cannot say in advance that a process will require a particular page at particular time.
- Therefore, to overcome this problem, there is a concept called Demand Paging is introduced. It suggests keeping all pages of the frames in the secondary memory until they are required (lazy sweeper)


program
A
program
B
main
memory
swap out
swap in
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23



Page table when some pages are not in main memory



Steps in handling a page fault

1. The memory address requested is first checked, to make sure it was a valid memory request.
2. If the reference was invalid,
3. A free frame is located, possibly from a free-frame list.
4. A disk operation is scheduled to bring in the necessary page from disk. (This will usually block the process on an I/O wait, allowing some other process to use the CPU in the meantime.)
5. When the I/O operation is complete, the process's page table is updated with the new frame number, and the invalid bit is changed to indicate that this is now a valid page reference.
6. The instruction that caused the page fault must now be restarted from the beginning, (as soon as this process gets another turn on the CPU.)

Performance of Demand Paging

- Page fault rate $0 \le p \le 1$
  - if $p = 0$, no page faults
  - if $p = 1$, every reference results in a page fault.
- Effective Access Time:

EAT =

(1 - p) effective_memory_access +

p (page fault overhead + [swap out] + swap in + restart overhead)

- suppose that a normal memory access requires 200 nanoseconds, and that servicing a page fault takes 8 milliseconds. With a ***page fault rate*** of p, (on a scale from 0 to 1), the effective access time is now:

$$(1 - p) * (200) + p * 8000000$$

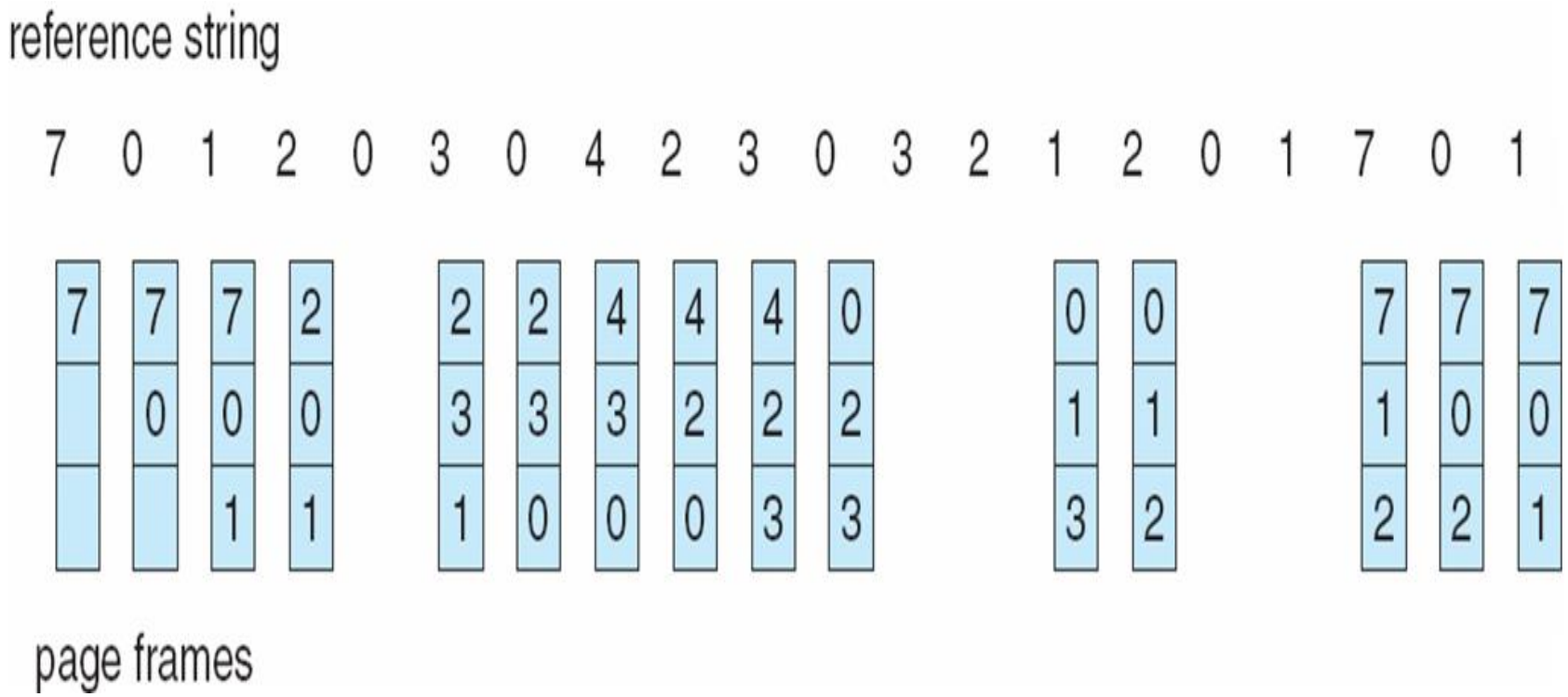
$$= 200 + 7{,}999{,}800 * p$$

Page Replacement Algorithms:

The FIFO Policy

- Treats page frames allocated to a process as a circular buffer:
  - When the buffer is full, the oldest page is replaced. Hence first-in, first-out:
    - A frequently used page is often the oldest, so it will be repeatedly paged out by FIFO.
  - Simple to implement:
    - requires only a pointer that circles through the page frames of the process.

- Given a reference string, and three frames are initially empty



First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process):

1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

- 4 frames:

1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

- FIFO Replacement manifests Belady's Anomaly:
  - more frames $\Rightarrow$ more page faults

Optimal Page replacement

- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

- page reference string 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Apply Optimal Page Replacement Algorithm to determine page fault.

PAGES	3	1	2	1	6	5	1	3
FRAMES			2	2	6	5	5	5
		1	1	1	1	1	1	1
	3	3	3	3	3	3	3	3
	Miss	Miss	Miss	Hit	Miss	Miss	Hit	Hit

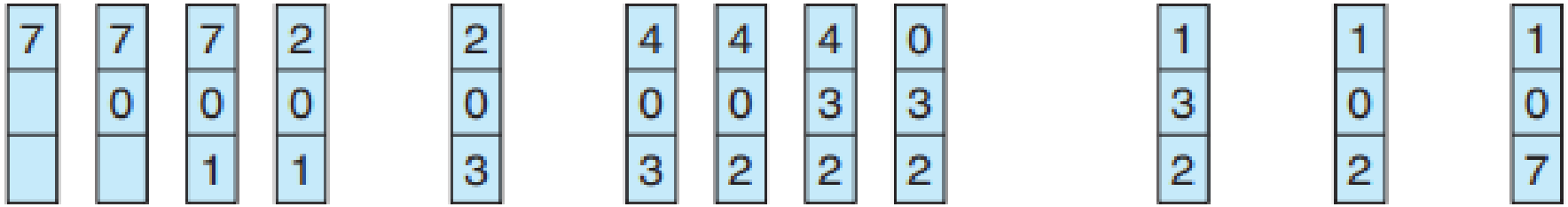
Total page faults = 5

Least Recently Used

- page will be replaced which is least recently used.

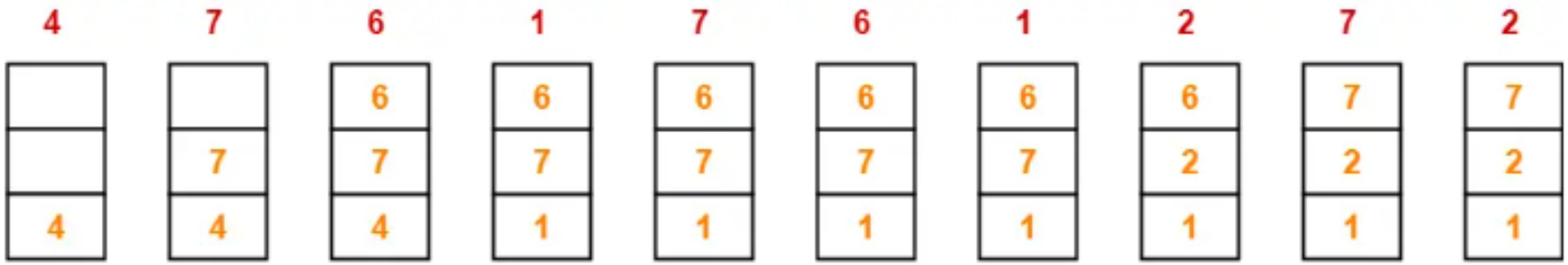
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

- A system uses 3 page frames for storing process pages in main memory. It uses the Least Recently Used (LRU) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below-
- 4 , 7, 6, 1, 7, 6, 1, 2, 7, 2
- Also calculate the hit ratio and miss ratio.
-



number of page faults occurred = 6

- **Hit ratio** = 0.4 or 40%
- **Miss ratio** = 0.6 or 60%

Determine the best page replacement algorithm among FIFI, OPR,LRU

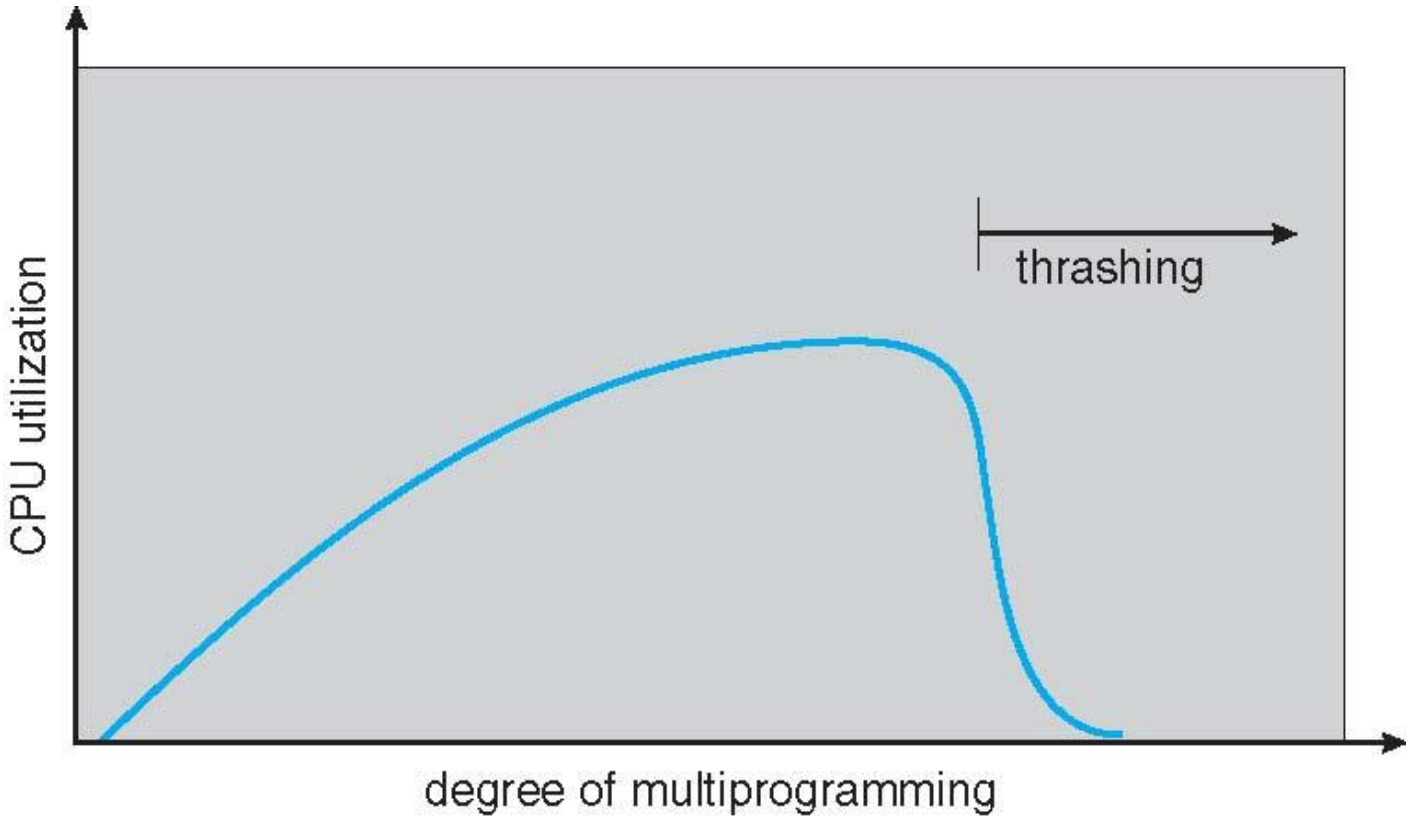
- Reference string: 0,2,1,6,4,0,1,0,3,1,2,1
- Frame size: 4

- **FIFO**: 9 (2,0,3,1)
- **OPR**: 6 (3,2,1,4)
- **LRU**: 8 (2,0,1,3)

Thrashing

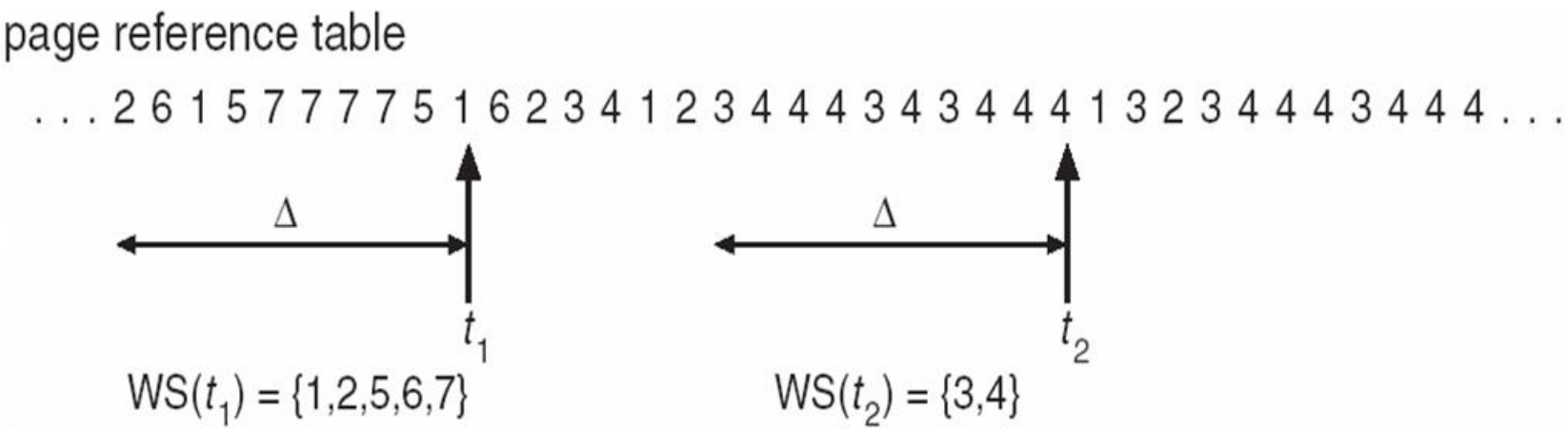
- If a process does not have “enough” pages, the page-fault rate is very high
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - Operating system thinking that it needs to increase the degree of multiprogramming
    - Another process added to the system
- **Thrashing** ≡ a process is busy swapping pages in and out

Thrashing (Cont.)



Working-Set Model

- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
  Example: 10,000 instructions
- WSS_i (working set of Process P_i) =
  total number of pages referenced in the most recent Δ (varies in time)
  - if Δ too small will not encompass entire locality
  - if Δ too large will encompass several localities
  - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \Sigma\ WSS_i \equiv$ total demand frames
  - Approximation of locality
- if $D > m \Rightarrow$ Thrashing
- Policy if $D >$ m, then suspend or swap out one of the processes

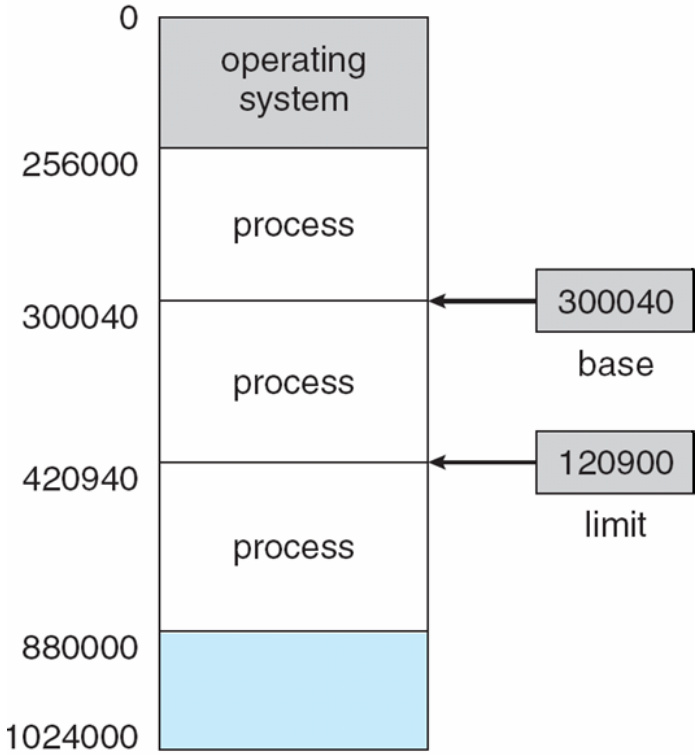


Main Memory

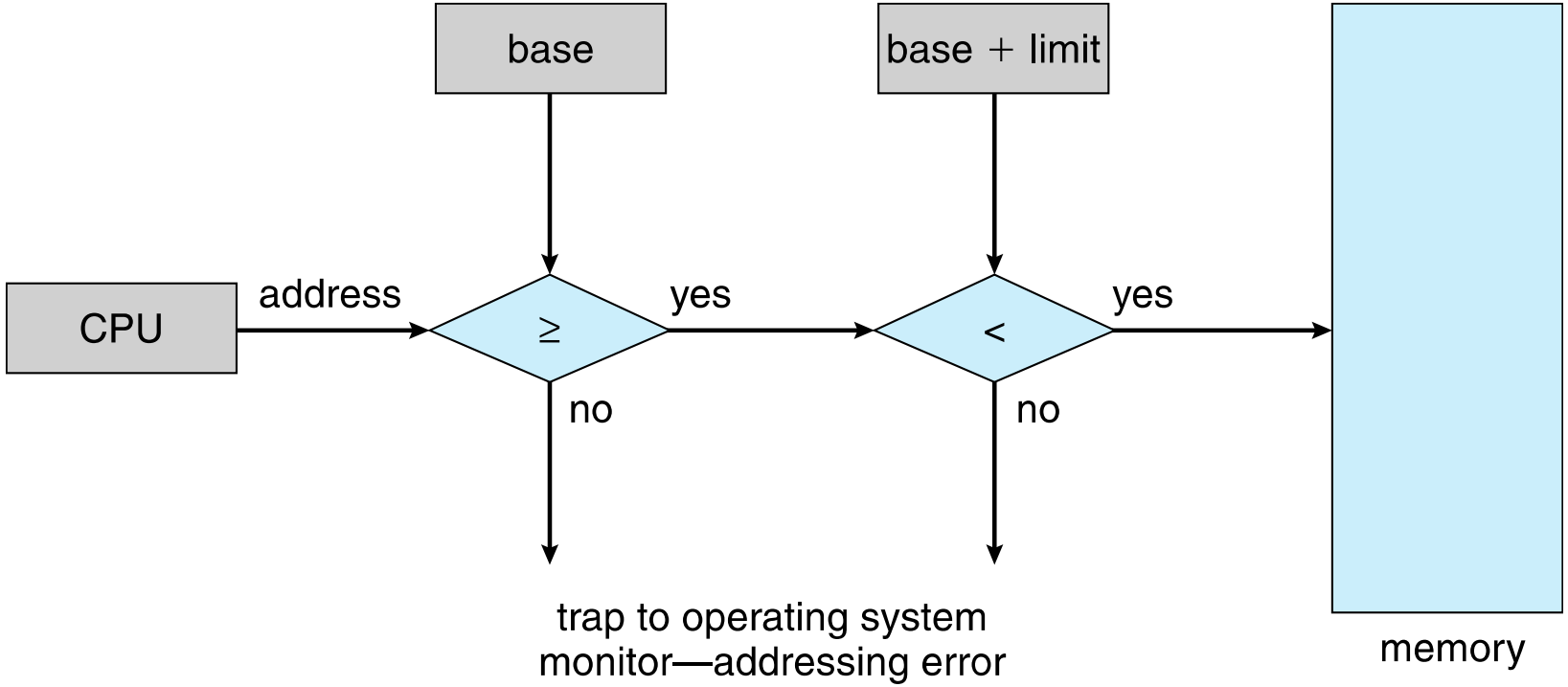
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers

Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user

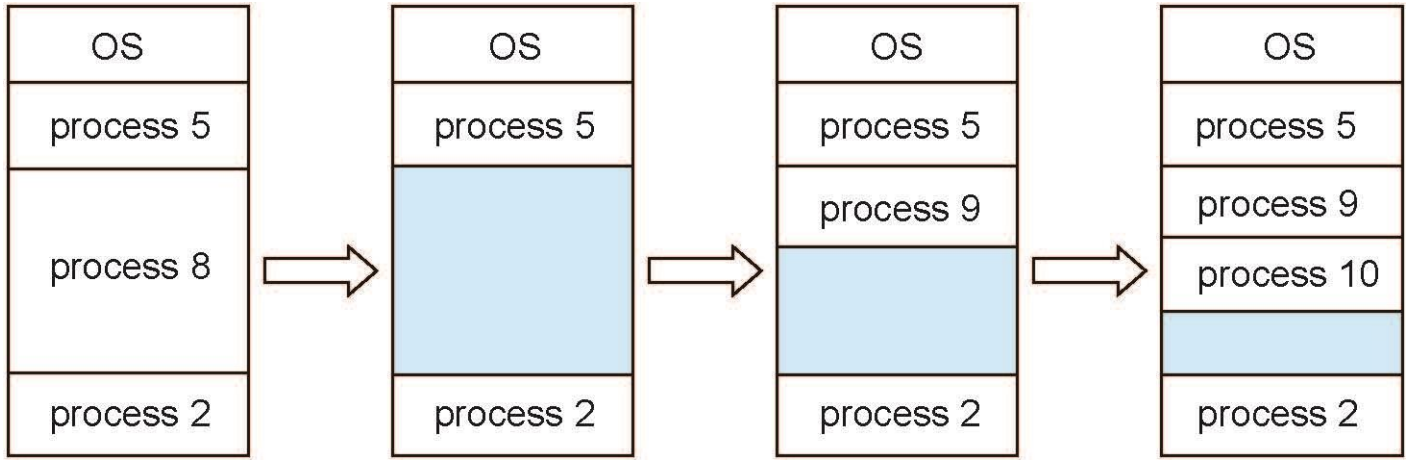


Hardware Address Protection



Multiple-partition allocation

- Multiple-partition allocation
  - Degree of multiprogramming limited by number of partitions
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)



Dynamic Storage-Allocation Problem

How to satisfy a request of size ***n*** from a list of free holes?

- **First-fit**: Allocate the ***first*** hole that is big enough
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization

- Consider five memory partitions of size 100 KB, 500 KB, 200 KB, 200 KB and 600 KB in same order. If sequence of requests for blocks of size 212 KB, 417 KB, 112 KB and 426 KB in same order come, then which of the following algorithm makes the efficient use of memory?